Übung 06 – 2. und 3. Normalform

1000 Datenbank · Teil 2 Normalisierung · ca. 45 Minuten · OneNote oder Excel



Luigi räumt auf

„Die Liste ist jetzt ordentlich – aber Bella Farinas Nummer steht immer noch in jeder Zeile. Wie kriegen wir das ganz sauber?“

Ziel	Funktionale Abhängigkeiten finden und Luigis Einkaufsliste über die 2NF in die 3NF zerlegen – wie auf den Folien mit Marios Liste.
Startpunkt	deine 1NF-Tabelle aus Übung 05 (Schlüssel: einkauf_nr, ware)
Folien	Präsentation „1000 Datenbank“ – vor der Übung erklärt und vorgemacht; bei jeder Aufgabe steht die passende Folie

Aufgabe 1 – Abhängigkeiten finden

Hilfe: *Präsentation „1000 Datenbank“, Folie 0460–0470 (Nummer unten rechts)*

Stelle für jede Spalte die Prüffrage: „Reicht `einkauf_nr`? Reicht `ware`? Oder braucht es beide?“

Spalte	Reicht einkauf_nr?	Reicht ware?	Ergebnis
datum	Ja	Nein	Partiell: einkauf_nr → datum
lieferant	Ja	Nein	Partiell: einkauf_nr → lieferant
telefon	Ja, über lieferant	Nein	Partiell und transitiv: einkauf_nr → lieferant → telefon
preis	Nein	Ja	Partiell: ware → preis
warengruppe	Nein	Ja	Partiell: ware → warengruppe
menge	Nein	Nein	Voll: (einkauf_nr, ware) → menge

Annahme im Grundmodell: preis ist der aktuelle Warenpreis, wie auf Folie 0470. Der Preis zum Einkaufszeitpunkt wird in der Zusatzaufgabe berücksichtigt.

Aufgabe 2 – In die 2NF zerlegen

Hilfe: *Präsentation „1000 Datenbank“, Folie 0480 (Nummer unten rechts)*

Zerlege so, dass es keine partiellen Abhängigkeiten mehr gibt. Schreibweise: `tabelle (spalte [PK], spalte [FK], …)`.

einkauf (einkauf_nr [PK], datum, lieferant, telefon)

ware (ware [PK], preis, warengruppe)

einkaufsposition (einkauf_nr [PK, FK], ware [PK, FK], menge)

Datum und Lieferant gehören zum Einkauf; Preis und Warengruppe zur Ware. Nur die Menge braucht den ganzen Positionsschlüssel. Über die Fremdschlüssel lassen sich die ursprünglichen Zeilen wieder zusammensetzen.

Aufgabe 3 – In die 3NF zerlegen

Hilfe: *Präsentation „1000 Datenbank“, Folie 0490 (Nummer unten rechts)*

1. Suche die transitiven Abhängigkeiten. Beispiel: `einkauf_nr → lieferant → telefon`. Findest du eine zweite?
2. Zerlege bis zur 3NF. Vergib künstliche Schlüssel (`lieferant_id`, `ware_id`, `warengruppe_id`, `einkauf_id`).
3. Vergleiche mit Marios Modell. Welche Tabelle entspricht welcher?

Transitiv: einkauf_nr → lieferant → telefon. Die Telefonnummer gehört deshalb in eine eigene Lieferantentabelle.

Zweite Kette mit Gruppen-IDs: ware_id → warengruppe_id → bezeichnung. Die Gruppenbezeichnung gehört in warengruppe. In der ursprünglichen 1NF-Tabelle ist ware → warengruppe bereits eine partielle Abhängigkeit, die in Aufgabe 2 ausgelagert wird.

lieferant (lieferant_id [PK], name, telefon)

warengruppe (warengruppe_id [PK], bezeichnung)

ware (ware_id [PK], name, preis, warengruppe_id [FK])

einkauf (einkauf_id [PK], einkauf_nr [UQ], datum, lieferant_id [FK])

einkaufsposition (einkauf_id [PK, FK], ware_id [PK, FK], menge)

Die bisherige einkauf_nr bleibt als eindeutige Geschäftsnummer erhalten (UQ = eindeutig); einkauf_id ist der neue künstliche Schlüssel.

Vergleich mit Mario: lieferant ↔ kunde; warengruppe ↔ kategorie; ware ↔ artikel; einkauf ↔ bestellung; einkaufsposition ↔ bestellposition.

Aufgabe 4 – Selbst üben: Marios Lieferdienst

Hilfe: *Präsentation „1000 Datenbank“, Folie 0470, 0500 (Nummer unten rechts)*

Mario will seine Lieferungen verwalten. Der Mitarbeiter hat diese Tabelle angelegt (`liefer_id` ist der Primärschlüssel); dazu kommt die Spalte `max_zuladung_kg` (Vespa 20, Lastenrad 80).

liefer_id	bestell_id	fahrer_id	fahrer_name	fahrer_telefon	kennzeichen	typ
L1	1001	F1	Luca Bianchi	+1 718 555 0201	NY-4471	Vespa
L2	1004	F1	Luca Bianchi	+1 718 555 0201	NY-4471	Vespa
L3	1005	F2	Rosa Greco	+1 718 555 0202	NY-9120	Lastenrad

1. Schreibe alle funktionalen Abhängigkeiten auf.
2. Ist die Tabelle in 2NF? Ist sie in 3NF? Begründe und zerlege in 3NF.

Zusatzspalte max_zuladung_kg: L1 = 20, L2 = 20, L3 = 80.

liefer_id → bestell_id, fahrer_id, fahrer_name, fahrer_telefon, kennzeichen, typ, max_zuladung_kg

fahrer_id → fahrer_name, fahrer_telefon

kennzeichen → typ; typ → max_zuladung_kg (Vespa: 20, Lastenrad: 80)

Damit gelten auch die transitiven Ketten liefer_id → fahrer_id → fahrer_name / fahrer_telefon und liefer_id → kennzeichen → typ → max_zuladung_kg.

2NF: Ja. Die Werte sind atomar und der angegebene Primärschlüssel liefer_id besteht aus einer Spalte. Es gibt keine partielle Abhängigkeit von einem Teil dieses Schlüssels.

3NF: Nein. Fahrername, Fahrertelefon, Fahrzeugtyp und maximale Zuladung hängen über andere Nichtschlüsselattribute vom Schlüssel ab. Zerlegung:

fahrer (fahrer_id [PK], fahrer_name, fahrer_telefon)

fahrzeugtyp (typ [PK], max_zuladung_kg)

fahrzeug (kennzeichen [PK], typ [FK])

lieferung (liefer_id [PK], bestell_id [FK], fahrer_id [FK], kennzeichen [FK])

bestell_id verweist auf Marios vorhandene Tabelle bestellung. Ein Fahrer kann das Fahrzeug wechseln; fahrer_id → kennzeichen wird deshalb nicht vorausgesetzt.

Das kann ich jetzt

- ☐ Ich finde partielle und transitive Abhängigkeiten mit der Prüffrage.
- ☐ Ich zerlege Tabellen verlustfrei in die 2NF und 3NF.
- ☐ Ich erkenne, dass Luigis Einkauf und Marios Bestellung gleich aufgebaut sind.

Zusatzaufgabe

Preis zum Einkaufszeitpunkt: Erhöht Bella Farina den Mehlpreis, ändern sich alte Einkäufe mit. Wie änderst du das Modell? Ist das noch 3NF?

Den tatsächlich bezahlten Einzelpreis beim Einkauf in der Position speichern:

einkaufsposition (einkauf_id [PK, FK], ware_id [PK, FK], menge, einzelpreis)

ware.preis bleibt der aktuelle Warenpreis. einzelpreis wird beim Einkauf übernommen und bei späteren Preisänderungen nicht verändert. So bleiben alte Einkaufssummen gleich.

Ja, das bleibt 3NF: Der historische Einzelpreis hängt von der gesamten Einkaufsposition ab, also von (einkauf_id, ware_id), und nicht allein von ware_id.

Zum Schluss: sichern mit Git

Speichere alles (Strg + S) und sichere deinen Stand im Repository `marios-pizza`:

```
git add .
git commit -m "Übung 06: Normalisierung 2NF und 3NF (Luigis Einkaufsliste)"
git push
```

Kontrolle: auf github.com das Repository neu laden – dein Commit und die neuen Dateien sind zu sehen.